Binary Search Trees

A binary search tree (BST) is a [[Binary Search|binary tree]] with the **search tree property**: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger.

Natural Explanation

The binary search tree property lets you search efficiently by comparing the target to the current node and deciding to go left (smaller) or right (larger).

Formal Definition

A [[Trees|binary tree]] T is a BST if:

For every node u: - All keys in the left subtree of u are strictly less than the key at u - All keys in the right subtree of u are strictly greater than the key at u

(Handling duplicates: some variants disallow; others allow in one direction consistently.)

Core Operations

SEARCH(key): - Start at root - If key equals current node, return found - If key is smaller, search left subtree - If key is larger, search right subtree - Depth-first search: $O(h)$ where h is tree height

INSERT(key, value): - Search to the position where key should go - Create new leaf node there - Maintains BST property if done correctly; $O(h)$

DELETE(key): - **Case 1** (leaf): remove it directly - **Case 2** (one child): replace node with its child - **Case 3** (two children): replace with inorder predecessor (largest in left subtree) or inorder successor (smallest in right subtree), then delete that node; $O(h)$

MIN/MAX: - MIN: follow left pointers until null; leftmost node - MAX: follow right pointers until null; rightmost node

---

Tree Height and Complexity

The worst case height is $n - 1$ (completely unbalanced, like a linked list); then all operations are $O(n)$.

Balanced BSTs (like [[Trees|AVL trees]] or red-black trees) maintain height $O(\log n)$, giving $O(\log n)$ operations.

Inorder Traversal Property

[[Tree Traversals|Inorder traversal]] of a BST visits nodes in **sorted order** (ascending). This property is useful for printing BST contents sorted or finding rank.

Predecessor and Successor

Inorder predecessor of node u: - If u has a left subtree, it is the rightmost (MAX) node in the left subtree - Otherwise, it is the lowest ancestor of u whose right child is an ancestor of u

Inorder successor of node u: - If u has a right subtree, it is the leftmost (MIN) node in the right subtree - Otherwise, it is the lowest ancestor of u whose left child is an ancestor of u

Both can be found in $O(h)$ time.

Use Cases

- **Sorted data**: efficient range queries and ranking
- **Dictionary/map**: when you need both fast lookup and sorted order
- **Building blocks for balanced BSTs**: every balanced tree is a BST

Related

- [[Trees]]
- [[Binary Search]]
- [[Tree Traversals]]
- [[Searching Algorithms]]